SHERLOCK

Security Review For Ozak AI



Collaborative Audit Prepared For:	**Ozak AI**
Lead Security Expert(s):	**0xeix**
Date Audited:	**September 10 - September 13, 2025**
Final Commit:	**dc30955**

Introduction

Ozak AI is an agentic AI platform that leverages predictive AI purpose-built for financial markets. By fusing state-of-the-art machine-learning models with decentralized infrastructure, Ozak AI delivers real-time, tamper-proof intelligence that helps individuals and institutions anticipate market shifts, mitigate risk, and unlock new trading strategies.

Scope

Repository: Ozak-AI/ozak-presale-contracts-foundry

Audited Commit: e42060e10219b5efcbae17709864196276d719fa

Final Commit: dc309553167cd9abbf55ac019972416515188324

Files:

- src/PresaleAdminAccess.sol
- src/RewardsRedeemer.sol
- src/TokenPresale.sol

Final Commit Hash

dc309553167cd9abbf55ac019972416515188324

Findings

Each issue has an assigned severity:

- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
3	1	7

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: Unauthorized access because of the wrongly assigned roles in the `PresaleAdminAccess`

Source: https://github.com/sherlock-audit/2025-09-ozak-ai-audit/issues/6

Summary

Some of the functions in the `PresaleAdminAccess` contract have wrongly set roles.

Vulnerability Detail

Let's take a look at the `updateMinMaxPurchase()` function implementation:

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/PresaleAdminAccess.sol#L189-201

```
/**
  * @notice Updates the minimum and maximum purchase amounts for the presale.
  * @dev This function can only be called by an address with the
   ↪  `PRESALE_CONTROLLER_ROLE`.
  * It directs the call to the underlying `tokenPresale` contract.
  * @param newMinPurchase The new minimum purchase amount.
  * @param newMaxPurchase The new maximum purchase amount.
  */
 function updateMinMaxPurchase(uint256 newMinPurchase, uint256 newMaxPurchase)
     external
     onlyRole(UPDATE_TOKEN_REFERRAL_ROLE)
 {
     tokenPresale.updateMinMaxPurchase(newMinPurchase, newMaxPurchase);
 }
```

As you can see here, it changes the presale related parameters such as `newMinPurchase` and `newMaxPurchase` and, logically (as stated in the comments as well), it has to be called by the `PRESALE_CONTROLLER_ROLE` but instead it's called by the `UPDATE_TOKEN_REFERRAL_ROLE`. Same thing can be observed in the `updateReferrerBonusPercentage()` function:

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/PresaleAdminAccess.sol#L117-125

```
/**
  * @notice Updates the referrer bonus percentage.
  * @dev This function can only be called by an address with the
   ↪  `UPDATE_TOKEN_REFERRAL_ROLE`.
  * It directs the call to the underlying `tokenPresale` contract.
  * @param newBonusPercentage The new bonus percentage for referrers.
  */
```

```
function updateReferrerBonusPercentage(uint256 newBonusPercentage) external
 ↪  onlyRole(PRESALE_CONTROLLER_ROLE) {
    tokenPresale.updateReferrerBonusPercentage(newBonusPercentage);
}
```

The function is said to be updated by the `UPDATE_TOKEN_REFERRAL_ROLE` but instead it can only be called by the `PRESALE_CONTROLLER_ROLE`.

Impact

Incorrect roles will call the functions.

Tool Used

Manual Review

Recommendation

Update the roles accordingly.

Issue H-2: Incorrect variable is used to calculate the `elapsedTime` parameter

Source: https://github.com/sherlock-audit/2025-09-ozak-ai-audit/issues/9

Summary

The parameter `elapsedTime` is calculated by using `vestingStartTime` instead of the `cliffEndTime`.

Vulnerability Detail

When calculating the `vestedTokens` we need to calculate the `elapsedTime` that's supposed to be a difference between the current timestamp and the `cliffEndTime`. However, the current implementation takes the `vestingStartTime` instead of the `cliffEndTime`:

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/TokenPresale.sol#L685

```
uint256 elapsedTime = block.timestamp - vestingStartTime; // Calculate elapsed time
↪  since cliff end
```

The parameter that's needed to be used is the following:

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/TokenPresale.sol#L676

```
// Calculate cliff end time
      uint256 cliffEndTime = vestingStartTime + cliffDuration;
```

Impact

Incorrect `elapsedTime` will lead to incorrect `vestedTokens` calculations.

Tool Used

Manual Review

Recommendation

Use the `cliffEndTime` instead of the `vestingStartTime` to calculate the `elapsedTime`.

Issue H-3: The purchase functions don't account for the possible different decimals and use the same `minPurchase` and `maxPurchase` amounts

Source: https://github.com/sherlock-audit/2025-09-ozak-ai-audit/issues/10

Summary

Possible different decimals are not accounted for the `usdAmount` in the `buyTokens` function leading to constant reverts if the whitelisted stablecoin with different number (than the one set in the `minPurchase` or ‘maxPurchase) of decimals is used.

Vulnerability Detail

The functionality of the `buyTokens()` function allows to use the stablecoins with different decimals:

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/TokenPresale.sol#L355-360

```
// For 6 Decimal Stable coins-
     // 6 decimals * 18 decimals / 6 decimals = 6 decimals

     // For 18 Decimal Stable coins-
     // 18 decimals * 18 decimals / 18 decimals
     uint256 tokenAmount = (usdAmount * 1 ether) / denominator; // Here make sure
      ↪  min purchase is above 1 usdt token
```

The transferred amount from the `msg.sender` is then equal to the `usdAmount` provided in the parameters of the function:

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/TokenPresale.sol#L336

```
function buyTokens(address tokenAddress, uint256 usdAmount, address referrer)
```

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/TokenPresale.sol#L387

```
// Transfer before state changes
IERC20Upgradeable(tokenAddress).safeTransferFrom(msg.sender, treasury, usdAmount);
```

However, the `minPurchase` and `maxPurchase` are always the same and set for a particular number of decimals meaning if the token has a different number of decimals, the function will constantly revert for that particular whitelisted stablecoin. In other words, if the `minPurchase` and `maxPurchase` are set for 6 decimals and the whitelisted stablecoin has 18 decimals, the check will be incorrect.

Impact

The `minPurchase` and `maxPurchase` checks currently don't work for the tokens with different decimals (like stablecoins with 6 or 18 decimal places).

Tool Used

Manual Review

Recommendation

Change the implementation so the approach to check the minimum and the maximum amount is unified for the stablecoins with different decimals.

Issue M-1: `stage.amountRaised` and `userData.usdInvested` are not normalized for the tokens with different decimals

Source: https://github.com/sherlock-audit/2025-09-ozak-ai-audit/issues/11

Summary

`buyTokens` function allows to purchase the tokens with different whitelisted stablecoins. However, the `usdInvested` and `amountRaised` parameters are only tied to a certain number of decimal places.

Vulnerability Detail

Let's take a look at how the `stage.amountRaised` and `userData.usdInvested` parameters are increased and calculated:

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/TokenPresale.sol#L516

```
stage.amountRaised += usdAmount;
```

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/TokenPresale.sol#L521

```
userData.usdInvested += usdAmount;
```

The corresponding `usdAmount` is taken from the parameters:

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/TokenPresale.sol#L333-336

```
* @param usdAmount The amount of Ether or ERC20 tokens being spent.
   * @param referrer The address of the referrer.
   */
  function buyTokens(address tokenAddress, uint256 usdAmount, address referrer)
```

Therefore, `usdAmount` can be a stablecoin with either 6 or 18 decimals:

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/TokenPresale.sol#L355-359

```
// For 6 Decimal Stable coins-
       // 6 decimals * 18 decimals / 6 decimals = 6 decimals

       // For 18 Decimal Stable coins-
```

```
        // 18 decimals * 18 decimals / 18 decimals
```

The problem is that `stage.amountRaised` and `userData.usdInvested` are not normalized and will reflect the incorrect aggregate amount. So, for instance, if the `usdAmount` is `100e6`, the parameters will be increased on `100e6` amount. But if the next stablecoin has 18 decimals, they will be again increased but this time on `100e18` token amount. Therefore, the aggregated value will be slightly bigger than `100e18`.

Impact

The aggregated value is not conversed to the single unit of measurement.

Tool Used

Manual Review

Recommendation

Change the implementation so the `amountRaised` and `usdInvested` can be compatible with different decimals (normalized to a single unit).

Issue L-1: Safe transfer functionality is not used

Source: https://github.com/sherlock-audit/2025-09-ozak-ai-audit/issues/4

Summary

Currently, the safe transfer functionality is not used when dealing with ERC20 tokens.

Vulnerability Detail

Let's take a look at the following instances:

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/RewardsRedeemer.sol#L182-185

```
} else {
        // if not ETH then its token
        IERC20(token).transferFrom(msg.sender, address(this), allocated);
    }
```

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/RewardsRedeemer.sol#L149-152

```
if (amountToSend > 0) {
         IERC20(token).transfer(msg.sender, amountToSend);
         emit Claimed(msg.sender, token, amountToSend);
     }
```

If we have a situation where the token returns a `false` boolean (the boolean value is unchecked in the `startRound()` function), the round will start anyway even if the tokens were technically not allocated.

Impact

Some operations in the `RewardsRedeemer` can start even if the token transfer was unsuccessful.

Tool Used

Manual Review

Recommendation

Use `SafeERC20` library to handle ERC20 tokens transfer logic.

Issue L-2: Incorrect role is assigned to the function in accordance with the comment

Source: https://github.com/sherlock-audit/2025-09-ozak-ai-audit/issues/5

Summary

At the moment, there's a mistake in the comments of the `withdrawAccidentalFunds()` function that states that a different role needs to call the function.

Vulnerability Detail

Let's take a look at the `withdrawAccidentalFunds()` implementation:

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/PresaleAdminAccess.sol#L269-277

```
/**
     * @notice Withdraws accidentally sent funds from the presale contract.
     * @dev This function can only be called by an address with the
     ↪  `PRESALE_CONTROLLER_ROLE`.
     * It directs the call to the underlying `tokenPresale` contract.
     * @param tokenAddress The address of the token to withdraw.
     */
    function withdrawAccidentalFunds(address tokenAddress) external
     ↪  onlyRole(UPDATE_TOKEN_REFERRAL_ROLE) {
        tokenPresale.withdrawAccidentalFunds(tokenAddress);
    }
```

The comment states that the function has to be called by the `PRESALE_CONTROLLER_ROLE` but, in fact, it's called by the `UPDATE_TOKEN_REFERRAL_ROLE`.

Impact

The comment is incorrect as has to be updated to improve the overall specification and avoid the confusion in the future.

Tool Used

Manual Review

Recommendation

Update the comment so that it reflects the correct information.

Issue L-3: Wrong modifier is used for the `buyTokens` function

Source: https://github.com/sherlock-audit/2025-09-ozak-ai-audit/issues/7

Summary

`buyTokens` function is used with the `onlyWhitelistedOrEther()` modifier even though it's supposed to be used only with ERC20 tokens.

Vulnerability Detail

As you can see from the code below, the function is used with a modifier `onlyWhitelistedOrEther()` that's supposed to check if the `tokenAddress` is either whitelisted ERC20 token or native ETH:

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/TokenPresale.sol#L330-344

```
/**
     * @dev Allows users to purchase tokens during the presale by sending Ether or
     ↪  whitelisted ERC20 tokens.
     * @param tokenAddress The address of the ERC20 token used for payment, or the
     ↪  zero address for Ether payment.
     * @param usdAmount The amount of Ether or ERC20 tokens being spent.
     * @param referrer The address of the referrer.
     */
    function buyTokens(address tokenAddress, uint256 usdAmount, address referrer)
        external
        payable
        nonReentrant
        onlyDuringPresale
        onlyWhenStageActive
        whenNotPaused
        onlyWhitelistedOrEther(tokenAddress)
    {
```

The problem is that it's supposed to be used only with ERC20 tokens so if the function is accidentally called with ETH (as allowed by the modifier), it'll revert as there's another function to handle ETH token purchases:

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/TokenPresale.sol#L492-499

```
function buyTokensETH(address referrer)
        external
```

```
        payable
        nonReentrant
        onlyDuringPresale
        onlyWhenStageActive
        whenNotPaused
        onlyWhitelistedOrEther(address(0))
```

Impact

Unexpected reverts if the wrong function is called.

Tool Used

Manual Review

Recommendation

Update the modifier that for the `buyTokens()` that only checks for the whitelisted ERC20 tokens.

Issue L-4: Missing validation for the `vestingDuration` and the `cliffDuration` in the constructor

Source: https://github.com/sherlock-audit/2025-09-ozak-ai-audit/issues/8

Summary

No validation is present in the constructor when setting the `vestingDuration` and the `cliffDuration` even though such validation is present in the functions that update these parameters.

Vulnerability Detail

Currently, `updateVestingDuration` and `updateCliffDuration` functions have the following validation to ensure that the `vestingDuration` is greater than the `cliffDuration`:

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/TokenPresale.sol#L242-246

```
function updateVestingDuration(uint256 newVestingDuration) external onlyOwner {
      require(newVestingDuration > cliffDuration, "Vesting !> cliff");
      vestingDuration = newVestingDuration;
      emit VestingParametersUpdated(newVestingDuration, cliffDuration);
  }
```

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/TokenPresale.sol#L248-256

```
/**
  * @dev Updates the cliff duration.
  * @param newCliffDuration The new cliff duration.
  */
 function updateCliffDuration(uint256 newCliffDuration) external onlyOwner {
     require(vestingDuration > newCliffDuration, "Vesting !> cliff");
     cliffDuration = newCliffDuration;
     emit VestingParametersUpdated(vestingDuration, newCliffDuration);
 }
```

The problem is that there's no such validation present in the constructor when setting the initial values:

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/TokenPresale.sol#L154-155

```
vestingDuration = _vestingDuration;
```

```
cliffDuration = _cliffDuration;
```

Impact

Insufficient validation can lead to wrongly set initial values.

Tool Used

Manual Review

Recommendation

Add the same validation to the constructor of the `TokenPresale` contract.

Issue L-5: Incorrect naming of the local variable

Source: https://github.com/sherlock-audit/2025-09-ozak-ai-audit/issues/12

Vulnerability Detail

The variable contains a typo mistake:

```
whistlistTokenCount += 1;
```

Impact

Overall readability suffers.

Tool Used

Manual Review

Recommendation

The variable (supposedly) should be named as `whitelistTokenCount`.

Issue L-6: Delisted payment tokens should be removed from the `tokenList` array

Source: https://github.com/sherlock-audit/2025-09-ozak-ai-audit/issues/13

Summary

When delisting the payment tokens, only the mapping is updated but the token is not removed from the `tokenList` array.

Vulnerability Detail

Consider the functionality of the `delistPaymentTokens()` function:

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/TokenPresale.sol#L309-318

```
/**
   * @dev Removes ERC20 tokens from the whitelist.
   * @param tokens An array of token addresses to delist.
   */
  function delistPaymentTokens(address[] calldata tokens) external onlyOwner {
      for (uint256 i = 0; i < tokens.length; i++) {
          whitelistedTokens[tokens[i]] = false;
          emit TokenDelisted(tokens[i]);
      }
  }
```

The function has to revert all the changes introduced in the `whitelistPaymentTokens()`:

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/TokenPresale.sol#L297-307

```
/**
   * @dev Whitelists a list of ERC20 tokens for payment in the presale.
   * @param tokens An array of token addresses to whitelist.
   */
  function whitelistPaymentTokens(address[] calldata tokens) external onlyOwner {
      for (uint256 i = 0; i < tokens.length; i++) {
          whitelistedTokens[tokens[i]] = true;
          tokenList.push(tokens[i]);
          emit TokenWhitelisted(tokens[i]);
      }
  }
```

As you can see, the new tokens are pushed to the storage `tokenList` array and the `getWhitelistedTokens()` later makes use of it by looping over this array:

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/TokenPresale.sol#L762-778

```
function getWhitelistedTokens() public view returns (address[] memory) {
     uint256 whistlistTokenCount = 0;
     for (uint256 i = 0; i < tokenList.length; i++) {
         if (whitelistedTokens[tokenList[i]] == true) {
             whistlistTokenCount += 1;
         }
     }
     address[] memory whitelistedList = new address[](whistlistTokenCount);
     uint256 whitelistIndex = 0;
     for (uint256 k = 0; k < tokenList.length; k++) {
         if (whitelistedTokens[tokenList[k]] == true) {
             whitelistedList[whitelistIndex] = tokenList[k];
             whitelistIndex += 1;
         }
     }
     return whitelistedList;
 }
```

Impact

The constantly growing array can makes the execution of the `getWhitelistedTokens()` more expensive every time (taking into account that this is the array in storage).

Code Snippet

Tool Used

Manual Review

Recommendation

Remove the tokens from the `tokenList` array if they were delisted.

Issue L-7: No check for insufficient balance when transferring tokens to the referrers

Source: https://github.com/sherlock-audit/2025-09-ozak-ai-audit/issues/14

Summary

At the moment, there's no check to ensure that the number of the tokens in the contract is enough for the referrers to claim their bonus.

Vulnerability Detail

When releasing the referrer bonus, there is no check to ensure that the balance of the contract will be enough for all the referrers to claim:

https://github.com/sherlock-audit/2025-09-ozak-ai-audit/blob/main/ozak-presale-contracts-foundry/src/TokenPresale.sol#L712-719

```
function releaseReferrerBonus() external onlyOwner {

    referrerBonusReleased = true;

    emit ReferrerBonusReleased();
}
```

Impact

Reverts for the referrers if the balance is not enough.

Tool Used

Manual Review

Recommendation

Introduce a check that makes sure that the referrers will have enough tokens to claim. Note that the tokens for the referrers are not subtracted during the purchases from the buyers' amount and are handled as separate additional tokens.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.